

Title: CPU Scheduling Algorithms to Find Turnaround Time and Waiting Time

Objective(s)

- To understand the concepts of preemptive and non-preemptive CPU scheduling algorithms.
- To implement First-Come, First-Served (FCFS) and Shortest Job First (SJF) scheduling algorithms.
- To implement Shortest Remaining Time Next (SRTN) as the preemptive variant of SJF.
- To implement Round Robin (RR) scheduling algorithm with configurable time quantum.
- To implement Lottery Scheduling and understand probabilistic process selection.
- To calculate waiting time, turnaround time, and average metrics for each algorithm.

Problem Analysis

CPU scheduling is a fundamental function of an operating system that determines which process in the ready queue is allocated the CPU at any given time. The scheduler must make efficient use of CPU time while maintaining fairness among competing processes. This lab explores five distinct scheduling policies, each with different trade-offs between simplicity, fairness, and performance.

Assume all processes are available for scheduling analysis. For preemptive algorithms, the scheduler may interrupt a running process when a new, higher-priority process arrives or when the time quantum expires. For non-preemptive algorithms, a process runs to completion once it is given the CPU.

1. First-Come, First-Served (FCFS) Scheduling Algorithm

FCFS is the simplest CPU scheduling algorithm. Processes are dispatched in the order in which they arrive in the ready queue. It is a non-preemptive algorithm: once a process begins execution, it runs until it completes or voluntarily releases the CPU.

For the FCFS algorithm, read the number of processes in the system and their CPU burst times. Schedule them in order of arrival. Calculate the waiting time and turnaround time for each process using the formulae below.

Waiting Time (WT) = Start Time - Arrival Time

Turnaround Time (TAT) = Completion Time - Arrival Time = Burst Time + Waiting Time

Consider the following example where all processes arrive at time 0:

Process	Burst Time (ms)
P1	24
P2	3

P3	3
----	---

If the processes arrive in the order P1, P2, P3 and are served in FCFS order, the Gantt chart would show P1 completing at 24 ms, P2 at 27 ms, and P3 at 30 ms. The average waiting time is $(0 + 24 + 27) / 3 = 17$ ms.

If the processes arrive in the order P2, P3, P1 instead, the average waiting time drops to $(0 + 3 + 6) / 3 = 3$ ms. This illustrates the convoy effect: a long process at the head of the queue significantly increases the waiting time for all subsequent processes.

2. Shortest Job First (SJF) Scheduling Algorithm

SJF is a non-preemptive algorithm that selects the process with the smallest CPU burst from the ready queue. If two processes have the same burst time, FCFS order is applied as a tie-breaker.

For the SJF algorithm, read the number of processes and their burst times, sort them in ascending order of burst time, and then schedule them. Calculate waiting time and turnaround time accordingly.

Consider the following example:

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

Using SJF scheduling, the execution order would be P4 (3 ms), P1 (6 ms), P3 (7 ms), P2 (8 ms). The average waiting time is $(3 + 16 + 9 + 0) / 4 = 7$ ms, compared to 10.25 ms under FCFS. SJF is provably optimal for minimising average waiting time among non-preemptive algorithms, but it can cause starvation of long processes if short processes keep arriving.

3. Shortest Remaining Time Next (SRTN) Scheduling Algorithm

SRTN is the preemptive version of SJF. Whenever a new process enters the ready queue, its burst time is compared with the remaining burst time of the currently running process. If the new process has a shorter remaining time, the running process is preempted and the new process gains the CPU.

For the SRTN algorithm, read the number of processes, their arrival times, and their CPU burst times. At each unit of time, select the process with the minimum remaining burst time. Calculate waiting time and turnaround time for each process.

Consider the following example:

Process	Arrival Time (ms)	Burst Time (ms)
---------	-------------------	-----------------

P1	0	8
P2	1	4
P3	2	9
P4	3	5

At time 0, P1 starts. At time 1, P2 arrives with burst 4, which is less than P1's remaining 7, so P1 is preempted. At time 2, P3 arrives with burst 9; P2 continues. At time 3, P4 arrives with burst 5; P2 (remaining 2) still runs. P2 finishes at time 5. P4 (remaining 5) then preempts P1 (remaining 7). P4 completes at time 10. P1 runs from 10 to 17. P3 runs from 17 to 26. The average waiting time is $(9 + 0 + 15 + 2) / 4 = 6.5$ ms, which is lower than non-preemptive SJF.

4. Round Robin (RR) Scheduling Algorithm

Round Robin is a preemptive algorithm designed specifically for time-sharing systems. Each process in the ready queue is assigned a fixed time interval called a time quantum (or time slice). The CPU scheduler cycles through the processes in circular order; if a process does not finish within its time quantum, it is preempted and moved to the back of the ready queue.

For the Round Robin algorithm, read the number of processes, their CPU burst times, and the time quantum size. Simulate execution in a circular queue: dequeue a process, run it for $\min(\text{remaining_burst}, \text{quantum})$ time units, then re-enqueue if it has remaining work. Calculate waiting time and turnaround time for each process.

Consider the following example with a time quantum of 4 ms:

Process	Burst Time (ms)
P1	24
P2	3
P3	3

The execution sequence would be: P1 (0-4), P2 (4-7), P3 (7-10), P1 (10-14), P1 (14-18), P1 (18-22), P1 (22-26), P1 (26-30). P2 finishes at 7 ms, P3 at 10 ms, P1 at 30 ms. The average waiting time is $(6 + 4 + 7) / 3 = 5.67$ ms. The performance of Round Robin depends heavily on the choice of time quantum. A very large quantum degenerates to FCFS; a very small quantum causes excessive context-switching overhead.

5. Lottery Scheduling Algorithm

Lottery Scheduling is a probabilistic scheduling algorithm. Each process is assigned a number of lottery tickets proportional to the share of CPU time it should receive. At each scheduling decision, the scheduler holds a lottery by drawing a random ticket number; the process holding that ticket wins the CPU for the next time quantum.

For the Lottery Scheduling algorithm, read the number of processes, their burst times, and the number of lottery tickets assigned to each process. At each time quantum, compute the total number of tickets held by all processes that still have remaining work, draw a random number in

that range, and select the winning process. Run it for one time quantum, then repeat. Calculate waiting time and turnaround time.

Consider the following example with a time quantum of 2 ms:

Process	Burst Time (ms)	Tickets Assigned
P1	10	10
P2	6	20
P3	4	5

Total tickets = 35. P2 holds about 57% of the tickets and is therefore expected to receive approximately 57% of CPU time. P1 holds 29%, and P3 holds 14%. Unlike deterministic algorithms, the exact completion order varies each run due to the random draw. Over many independent runs, the average turnaround times will converge toward proportional values. Lottery scheduling provides a simple mechanism for proportional-share resource allocation without requiring complex priority queues.

Input / Output

The following subsections specify the expected input format and sample output for each scheduling algorithm.

FCFS

Input: Number of processes (n), followed by n burst times.

Output: Process ID, Burst Time, Waiting Time, Turnaround Time for each process, and the average waiting time and average turnaround time.

Process	Burst Time (ms)	Waiting Time (ms)	Turnaround Time (ms)
P1	24	0	24
P2	3	24	27
P3	3	27	30

SJF

Input: Number of processes (n), followed by n burst times.

Output: Processes sorted by burst time, with waiting time and turnaround time for each.

Process	Burst Time (ms)	Waiting Time (ms)	Turnaround Time (ms)
---------	-----------------	-------------------	----------------------

P4	3	0	3
P1	6	3	9
P3	7	9	16
P2	8	16	24

SRTN

Input: Number of processes (n), each with an arrival time and burst time.

Output: Completion time, waiting time, and turnaround time for each process.

Process	Arrival (ms)	Burst (ms)	Completion (ms)	Waiting (ms)	Turnaround (ms)
P1	0	8	17	9	17
P2	1	4	5	0	4
P3	2	9	26	15	24
P4	3	5	10	2	7

Round Robin

Input: Number of processes (n), burst times, and time quantum (q).

Output: Execution order, completion time, waiting time, and turnaround time for each process.

Process	Burst Time (ms)	Waiting Time (ms)	Turnaround Time (ms)
P1	24	6	30
P2	3	4	7
P3	3	7	10

Lottery Scheduling

Input: Number of processes (n), burst times, tickets per process, and time quantum (q).

Output: Execution log showing the ticket draw at each scheduling point, and the resulting waiting time and turnaround time.

Note: Results are non-deterministic. A sample run is shown below; actual output will vary.

Process	Burst (ms)	Tickets	Completion (ms)	Waiting (ms)	Turnaround (ms)
P1	10	10	18	8	18

P2	6	20	12	6	12
P3	4	5	20	16	20

Lab Task and Lab Report (Please implement yourself and show the output to the instructor)

- Implement the First-Come, First-Served (FCFS) CPU scheduling algorithm. Test it with at least five processes of varying burst times.
- Implement the Shortest Job First (SJF) CPU scheduling algorithm. Demonstrate the difference in average waiting time compared to FCFS using the same process set.
- Implement the Shortest Remaining Time Next (SRTN) algorithm. Use a process set where processes arrive at different times to demonstrate preemption.
- Implement the Round Robin (RR) CPU scheduling algorithm. Test it with two different values of time quantum and compare the results.
- Implement the Lottery Scheduling algorithm. Run it three times with the same input and record the different outcomes to illustrate the probabilistic nature of the algorithm.